

Codings for mini project

```
Package SmartInventoryH2.java;
```

```
Import javax.swing.*;
```

```
Import javax.swing.table.DefaultTableModel;
```

```
Import java.awt.*;
```

```
Import java.awt.event.*;
```

```
Import java.sql.*;
```

```
Import java.util.Vector;
```

```
Public class SmartInventoryH2 extends JFrame {
```

```
    Private Connection conn;
```

```
    Private JTextField txtId, txtName, txtQty, txtPrice, txtSearch;
```

```
    Private JTable table;
```

```
    Private DefaultTableModel model;
```

```
    Public SmartInventoryH2() {
```

```
        setTitle("SmartInventory H2 – Embedded DB");
```

```
        setSize(800, 500);
```

```
        setDefaultCloseOperation(EXIT_ON_CLOSE);
```

```
        setLocationRelativeTo(null);
```

```
        setLayout(new BorderLayout(10, 10));
```

```
        initDB(); // Creates DB in memory or file
```

```
        initUI();
```

```
        loadData();
```

```
}
```

```
Private void initDB() {
```

```
    Try {
```

```
        // EMBEDDED MODE: Saves to file in project folder
```

```
        Conn = DriverManager.getConnection(
```

```
            "jdbc:h2:./inventory_db", "sa", "");
```

```
        // Create table if not exists
```

```
        Statement s = conn.createStatement();
```

```
        s.execute("""
```

```
            CREATE TABLE IF NOT EXISTS products (
```

```
                Id INT AUTO_INCREMENT PRIMARY KEY,
```

```
                Name VARCHAR(100) NOT NULL,
```

```
                Quantity INT NOT NULL CHECK (quantity >= 0),
```

```
                Price DECIMAL(10,2) NOT NULL CHECK (price > 0),
```

```
                Last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
            )
```

```
            """);
```

```
        // Insert sample data only if table is empty
```

```
        ResultSet rs = s.executeQuery("SELECT COUNT(*) FROM products");
```

```
        Rs.next();
```

```
        If (rs.getInt(1) == 0) {
```

```
            s.execute("""
```

```
                INSERT INTO products (name, quantity, price) VALUES
```

```

        ('Dell Laptop', 8, 850.00),
        ('Wireless Mouse', 45, 25.50),
        ('Mechanical Keyboard', 3, 120.00)
        """);
    }
} catch (SQLException e) {
    JOptionPane.showMessageDialog(this,
        "H2 DB Error: " + e.getMessage(), "Error",
        JOptionPane.ERROR_MESSAGE);
    System.exit(0);
}
}

```

```

Private void initUI() {
    JPanel top = new JPanel(new BorderLayout());
    JPanel searchPanel = new JPanel();
    searchPanel.add(new JLabel("Search:"));
    txtSearch = new JTextField(20);
    JButton btnSearch = new JButton("Search");
    searchPanel.add(txtSearch);
    searchPanel.add(btnSearch);
    top.add(searchPanel, BorderLayout.NORTH);

    JPanel input = new JPanel(new GridLayout(4, 2, 10, 10));
    Input.setBorder(BorderFactory.createTitledBorder(" Product Details "));
    Input.add(new JLabel(" ID (for update/delete):"));
}

```

```
Input.add(txtId = new JTextField());  
Input.add(new JLabel(" Name:"));  
Input.add(txtName = new JTextField());  
Input.add(new JLabel(" Quantity:"));  
Input.add(txtQty = new JTextField());  
Input.add(new JLabel(" Price:"));  
Input.add(txtPrice = new JTextField());  
Top.add(input, BorderLayout.CENTER);
```

```
JPanel buttons = new JPanel();  
JButton[] btns = {  
    New JButton("Add"), new JButton("Update"),  
    New JButton("Delete"), new JButton("Clear"), new JButton("Refresh")  
};  
For (JButton b : btns) buttons.add(b);  
Top.add(buttons, BorderLayout.SOUTH);  
Add(top, BorderLayout.NORTH);
```

```
Table = new JTable();  
Table.setRowHeight(25);  
Add(new JScrollPane(table), BorderLayout.CENTER);
```

```
// Listeners
```

```
Btns[0].addActionListener(e -> addProduct());  
Btns[1].addActionListener(e -> updateProduct());  
Btns[2].addActionListener(e -> deleteProduct());
```

```

Btns[3].addActionListener(e -> clear());

Btns[4].addActionListener(e -> loadData());

btnSearch.addActionListener(e -> search());

txtSearch.addKeyListener(new KeyAdapter() {
    public void keyReleased(KeyEvent e) { search(); }
});

Table.addMouseListener(new MouseAdapter() {
    Public void mouseClicked(MouseEvent e) {
        Int row = table.getSelectedRow();
        If (row >= 0 && model != null) {
            txtId.setText(model.getValueAt(row, 0).toString());
            txtName.setText(model.getValueAt(row, 1).toString());
            txtQty.setText(model.getValueAt(row, 2).toString());
            txtPrice.setText(model.getValueAt(row, 3).toString());
        }
    }
});
}

Private void loadData() {
    Try {
        Model = new DefaultTableModel();
        Statement s = conn.createStatement();
        ResultSet rs = s.executeQuery("SELECT * FROM products");
        ResultSetMetaData md = rs.getMetaData();
    }
}

```

```

        For (int i = 1; i <= md.getColumnCount(); i++)
            Model.addColumn(md.getColumnName(i));
        While (rs.next()) {
            Vector<Object> row = new Vector<>();
            For (int i = 1; i <= md.getColumnCount(); i++)
                Row.add(rs.getObject(i));
            Model.addRow(row);
        }
        Table.setModel(model);
    } catch (Exception e) { showError€; }
}

```

```

Private void addProduct() {
    If (!validateInput()) return;
    String sql = "INSERT INTO products (name, quantity, price) VALUES (?, ?, ?)";
    executeUpdate(sql, false);
}

```

```

Private void updateProduct() {
    If (txtId.getText().trim().isEmpty()) {
        JOptionPane.showMessageDialog(this, "Select or enter ID!");
        Return;
    }
    If (!validateInput()) return;
    String sql = "UPDATE products SET name=?, quantity=?, price=? WHERE id=?";
    executeUpdate(sql, true);
}

```

```
}
```

```
Private void deleteProduct() {
```

```
    If (txtId.getText().trim().isEmpty()) {
```

```
        JOptionPane.showMessageDialog(this, "Enter ID!");
```

```
        Return;
```

```
    }
```

```
    If (JOptionPane.showConfirmDialog(this, "Delete ID " + txtId.getText() + "?", "Confirm",
```

```
        JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
```

```
    String sql = "DELETE FROM products WHERE id = ?";
```

```
    Try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```
        Ps.setInt(1, Integer.parseInt(txtId.getText()));
```

```
        If (ps.executeUpdate() > 0) {
```

```
            JOptionPane.showMessageDialog(this, "Deleted!");
```

```
            Clear(); loadData();
```

```
        } else {
```

```
            JOptionPane.showMessageDialog(this, "Not found!");
```

```
        }
```

```
    } catch (Exception e) { showError€; }
```

```
}
```

```
Private void search() {
```

```
    String term = txtSearch.getText().trim();
```

```
    String sql = "SELECT * FROM products WHERE name LIKE ?";
```

```
    Try (PreparedStatement ps = conn.prepareStatement(sql)) {
```

```

        Ps.setString(1, "%" + term + "%");
        updateTable(ps.executeQuery());
    } catch (Exception e) { showError€; }
}

```

```

Private void executeUpdate(String sql, boolean isUpdate) {
    Try (PreparedStatement ps = conn.prepareStatement(sql)) {
        Ps.setString(1, txtName.getText());
        Ps.setInt(2, Integer.parseInt(txtQty.getText()));
        Ps.setDouble(3, Double.parseDouble(txtPrice.getText()));
        If (isUpdate) ps.setInt(4, Integer.parseInt(txtId.getText()));

        Ps.executeUpdate();

        JOptionPane.showMessageDialog(this, isUpdate ? "Updated!" : "Added!");
        Clear(); loadData();

        Int qty = Integer.parseInt(txtQty.getText());
        If (qty < 5) {
            JOptionPane.showMessageDialog(this,
                "LOW STOCK: Only " + qty + " left!", "Alert",
                JOptionPane.WARNING_MESSAGE);
        }
    } catch (Exception e) { showError€; }
}

```

```

Private boolean validateInput() {

```



```

If (txtName.getText().trim().isEmpty()) {
    JOptionPane.showMessageDialog(this, "Name required!");
    Return false;
}
Try {
    Int q = Integer.parseInt(txtQty.getText());
    Double p = Double.parseDouble(txtPrice.getText());
    If (q < 0 || p <= 0) throw new Exception();
} catch (Exception e) {
    JOptionPane.showMessageDialog(this, "Invalid quantity or price!");
    Return false;
}
Return true;
}

```

```

Private void updateTable(ResultSet rs) throws SQLException {
    Model = new DefaultTableModel();
    ResultSetMetaData md = rs.getMetaData();
    For (int l = 1; l <= md.getColumnCount(); l++)
        Model.addColumn(md.getColumnName(l));
    While (rs.next()) {
        Vector<Object> row = new Vector<>();
        For (int l = 1; l <= md.getColumnCount(); l++)
            Row.add(rs.getObject(l));
        Model.addRow(row);
    }
}

```

```
Table.setModel(model);  
}
```

```
Private void clear() {  
    txtId.setText(""); txtName.setText("");  
    txtQty.setText(""); txtPrice.setText("");  
    txtSearch.setText("");  
}
```

```
Private void showError(Exception e) {  
    JOptionPane.showMessageDialog(this, e.getMessage(), "Error",  
        JOptionPane.ERROR_MESSAGE);  
}
```

```
Public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> new SmartInventoryH2().setVisible(true));  
}  
}
```